Application of stochastic character mapping to reconstruct the origin of somatic mutations

Michael Tassia

2026-02-02

Introduction

To infer the origin of somatic mutations using stochastic character mapping (SCM; algorithm description can be found in Bollback 2006), we use the comparative phylogenetic toolkit `phytools` in R. Comparative phylogenetics has a long history within R, and the modular design of `phytools` (and its dependencies) make it simple to implement SCM for a single somatic single nucleotide variant (SNV).

Methods

First, the following libraries are required for loading data and performing SCM. These libraries are included in the conda environment specified in `scm_env.yaml`.

```
# Required packages
library(phytools)
library(tidyverse)
library(data.table)
library(vcfR)
library(pbapply)
library(snow)
library(RColorBrewer)
```

Additionally, we will load custom functions for reading data and performing SCM from the `scm_functions.R` script.

```
source("scm_functions.R")
```

Load data

There are three fundamental parameters to SCM:

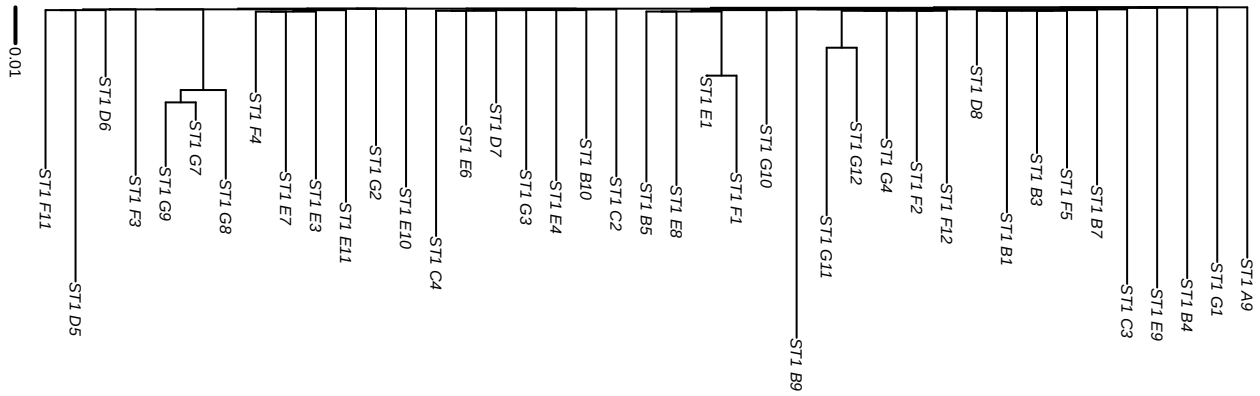
1. A phylogenetic tree describing the relationship among all samples (cells), with branch lengths measured in substitutions per site.
2. Substitution matrix ($\mathbf{Q}$) describing the rates of change between character states (e.g., unphased genotype states), jointly inferred with the phylogenetic tree (see CellPhy manuscript)
3. Genotype state priors for each sample (cell)

We will load (and visualize) each of these components below.

Phylogenetic tree The phylogeny needs to be rooted such that the root node best reflects the germline state; this allows for empirical priors for the root node if germline states are known and/or uncertain. Here, we use a matched normal (`OUTGROUP`) as the outgroup to define the root.

```r
# Read tree
tree <- read.tree(file = "data/ST1.nwk",) %>%
  root(outgroup = "OUTGROUP", resolve.root = TRUE) %>%
  drop.tip("OUTGROUP")

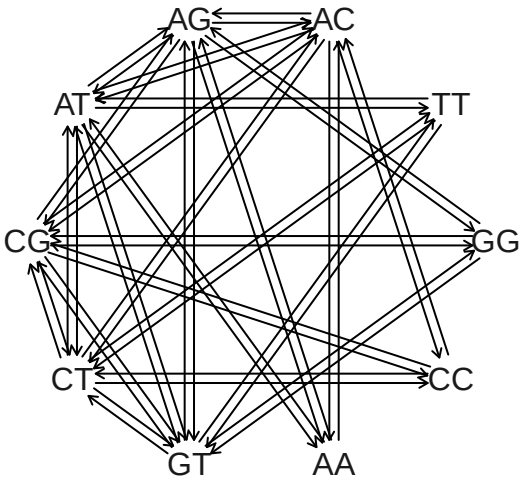
# Plot tree with scalebar (sub/site)
plot.phylo(x = ladderize(tree),
         direction = "downwards",
         color = "black",
         cex = 0.5,
         fsize = 0.8,
         lwd = 2)
add.scale.bar(length = 0.01,
              lwd = 2,
              x = 0,
              y = max(nodeHeights(tree)), cex = 0.5)
```



Substitution model The substitution model file output by CellPhy contains information on the relative rates of change between unphased diploid genotypes (A/A, A/C, A/G, A/T, C/C, C/G, C/T, G/G, G/T, T/T) and the relative frequencies for each genotype state. The formatting for this file (as output by `CellPhy` version `0.9.2`) is discussed in this thread.

```r
# Read substitution model
q <- read_cellphy_model("data/ST1.Q.txt")

# For visualization purposes, we will replace all "near-zero" values with zeroes
# and remove all vectors with zero rates
q.plot <- q
q.plot[which(q.plot == 0.001)] <- 0
plot.Qmatrix(x = q.plot,
             show.zeros = FALSE,
             text = FALSE,
             signif = 2)
```



Genotype state priors Next we will load a multi-sample VCF of somatic variants, where all tips in `tree` are represented as samples in the VCF. From this VCF, we can compute scaled probabilities from the Phred-scaled genotype likelihoods (`PL` FORMAT field). Here, we are using a small subset of variants for demonstration purposes only (these are somatic variants with > 1 origin on the tree).

```
# Read VCF
vcf <- read.vcfR("data/ST1.scm_vars.vcf.gz", verbose = FALSE)

# Compile somatic SNV genotype priors
gt_states <- compile_gt_states.snp(vcf)
```

```
## [1] "Reading vcf data..."
```

```
## [1] "Computing genotype probabilities..."
## [1] "Assembling genotype state matrices for SNPs..."
```

The function `compile_gt_states.snp()` reads the `vcfR` object and outputs a list of of data.tables, where each data.table can be accessed either through list indexing (e.g., `gt_states[[1]]`). Alternatively, an index can be accessed by its `<chrom>_<pos>` string:

```
tibble(gt_states[["chr9_125226000"]])
```

```
## # A tibble: 41 x 12
##    locus      clone       AA    CC    GG       TT    AC    AG       AT    CG    CT
##    <chr>      <chr>    <dbl> <dbl> <dbl>    <dbl> <dbl> <dbl>    <dbl> <dbl> <dbl>
##  1 chr9_1252~ ST1_~ 6.30e-28     0     0 9.98e-1     0     0 1.99e-3     0     0
##  2 chr9_1252~ ST1_~ 2.51e- 7     0     0 1.58e-8     0     0 1.00e+0     0     0
##  3 chr9_1252~ ST1_~ 6.31e-67     0     0 1.00e+0     0     0 1.00e-6     0     0
##  4 chr9_1252~ ST1_~ 3.16e-95     0     0 1.00e+0     0     0 5.01e-7     0     0
##  5 chr9_1252~ ST1_~ 7.91e-28     0     0 9.96e-1     0     0 3.97e-3     0     0
##  6 chr9_1252~ ST1_~ 3.16e-77     0     0 1.00e+0     0     0 7.94e-6     0     0
##  7 chr9_1252~ ST1_~ 3.97e-28     0     0 9.98e-1     0     0 1.99e-3     0     0
##  8 chr9_1252~ ST1_~ 3.98e-30     0     0 9.99e-1     0     0 9.99e-4     0     0
##  9 chr9_1252~ ST1_~ 1.58e-50     0     0 1.00e+0     0     0 6.31e-5     0     0
## 10 chr9_1252~ ST1_~ 1.00e-30     0     0 1.00e-6     0     0 1.00e+0     0     0
## # i 31 more rows
## # i 1 more variable: GT <dbl>
```

Perform SCM

To perform SCM for a single somatic SNP, we will use the wrapper function `runSCM_single()`. In this example here, we will run SCM for 1000 replicates on the locus `chr9_125226000` using a naive root state prior.

```
scm <- runSCM_single(locus_str = "chr9_125226000",
              tree = tree,
              gt_state_list = gt_states,
              Qmat = q,
              reps = 1000,
              cores = 6)
```

```
## [1] "Performing stochastic character mapping on chr9_125226000."
## [1] "Cores: 6"
## [1] "Replicates: 1000"
## [1] "Reduced Qmatrix: FALSE"
```

Because we requested 1000 replicates and 6 cores, the wrapper rounded up to the nearest multiple of 6 (i.e., 1002 replicates) and distributed the SCM replicates across 6 parallel R processes. The wrapper runs N instances of the `phytools` function `make.simmap()`, where N is the number of requested cores.

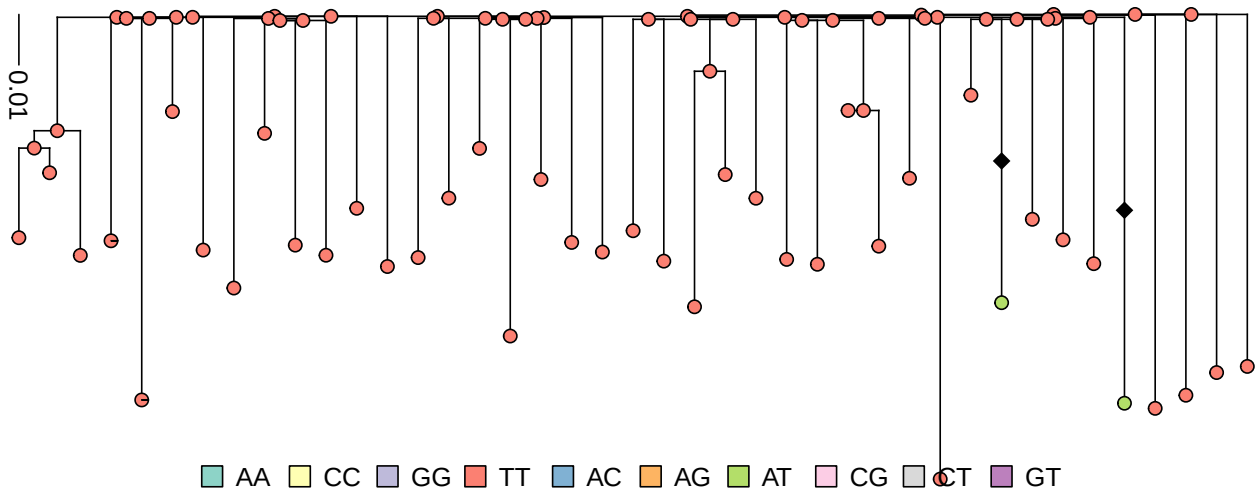
Below, we will summarize across the posterior distribution to infer the number of state changes on the tree:

```
summary(scm)
```

```
## 1002 trees with a mapped discrete character with states:
##  AT, TT
##
## trees have 2.01097804391218 changes between states on average
##
## changes are of the following types:
##      AT,TT   TT,AT
## x->y     0 2.00499
##
## mean total time spent in each state is:
##              AT       TT    total  AA  CC  GG  AC  AG  CG           CT  GT
## raw  0.06482629 1.783428 1.848295   0   0   0   0   0   0 4.069343e-05   0
## prop        Inf      Inf      Inf NaN NaN NaN NaN NaN NaN          NaN NaN
```

Finally, we can plot state transitions (as black diamonds) along edges where the genotype states shift from ancestor to descendent and both genotype calls have posterior probabilities ≥ 0.95.

```
scm_summary <- summarise_scm.snp(scm,
                                 plot = T,
                                 legend = T,
                                 PPthreshold = 0.95)
```



Environment

```
sessionInfo()
```

```
## R version 4.3.0 (2023-04-21)
## Platform: aarch64-apple-darwin20 (64-bit)
## Running under: macOS 14.6.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] RColorBrewer_1.1-3 snow_0.4-4         pbapply_1.7-2      vcfR_1.15.0
##  [5] data.table_1.16.4  lubridate_1.9.4    forcats_1.0.0      stringr_1.5.1
##  [9] dplyr_1.1.4        purrr_1.0.2        readr_2.1.5        tidyr_1.3.1
## [13] tibble_3.2.1       ggplot2_3.5.2      tidyverse_2.0.0    phytools_2.4-4
## [17] maps_3.4.2.1       ape_5.8-1
##
## loaded via a namespace (and not attached):
##  [1] fastmatch_1.1-6         gtable_0.3.6            xfun_0.50
##  [4] lattice_0.22-6          tzdb_0.4.0              numDeriv_2016.8-1.1
##  [7] quadprog_1.5-8          vctrs_0.6.5             tools_4.3.0
## [10] generics_0.1.3          parallel_4.3.0          cluster_2.1.8
## [13] pkgconfig_2.0.3         Matrix_1.6-5            scatterplot3d_0.3-44
## [16] lifecycle_1.0.4         compiler_4.3.0          farver_2.1.2
## [19] tinytex_0.54            mnormt_2.1.1            combinat_0.0-8
## [22] codetools_0.2-20        permute_0.9-7           htmltools_0.5.8.1
## [25] yaml_2.3.10             pillar_1.10.1           MASS_7.3-60.0.1
```

```
## [28] clusterGeneration_1.3.8 vegan_2.6-8            iterators_1.0.14
## [31] foreach_1.5.2           nlme_3.1-166           phangorn_2.12.1
## [34] tidyselect_1.2.1        digest_0.6.37          stringi_1.8.4
## [37] splines_4.3.0           fastmap_1.2.0          grid_4.3.0
## [40] expm_1.0-0              cli_3.6.3              magrittr_2.0.3
## [43] utf8_1.2.4              dichromat_2.0-0.1      optimParallel_1.0-2
## [46] withr_3.0.2             scales_1.4.0           DEoptim_2.2-8
## [49] timechange_0.3.0        rmarkdown_2.29         igraph_2.1.2
## [52] hms_1.1.3               memuse_4.2-3           coda_0.19-4.1
## [55] evaluate_1.0.3          knitr_1.49             doParallel_1.0.17
## [58] viridisLite_0.4.2       mgcv_1.9-1             rlang_1.1.4
## [61] Rcpp_1.0.14             pinfsc50_1.3.0         glue_1.8.0
## [64] rstudioapi_0.17.1       R6_2.5.1
```